



Apache Airflow

What is a Data Pipeline?

A **data pipeline** is a series of automated steps that move data from a source to a destination, performing various operations along the way. Think of it as an assembly line for data—raw information enters at one end, undergoes multiple transformations, and emerges as clean, structured, and ready-to-use data at the other end.

Key Characteristics:

- **Automated:** Once configured, pipelines run without manual intervention
- **Repeatable:** The same steps execute consistently on each run
- **Cyclic:** Pipelines typically run on schedules (hourly, daily, weekly)
- **Reliable:** Built-in error handling and retry mechanisms ensure data delivery

The Three Core Stages

Every data pipeline consists of three fundamental stages:

Stage 1: Data Extraction (Ingestion): The process of retrieving data from various sources:

- Databases (PostgreSQL, MySQL, MongoDB)
- APIs (REST, GraphQL, SOAP)
- Files (CSV, JSON, Parquet, Avro)
- Streaming sources (Kafka, Kinesis)
- Log files and system events
- Cloud storage (S3, Azure Blob, GCS)



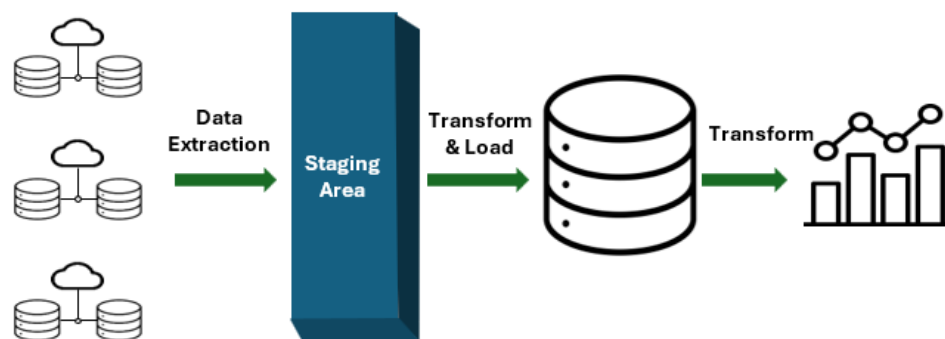
Stage 2: Data Transformation (Processing): The critical middle layer where raw data becomes valuable:

- **Cleaning:** Removing duplicates, handling nulls, correcting errors
- **Filtering:** Selecting relevant data, excluding outliers
- **Enrichment:** Joining data from multiple sources, adding calculated fields
- **Aggregation:** Computing totals, averages, counts
- **Normalization:** Standardizing formats, units, and schemas
- **Validation:** Ensuring data quality through checks and constraints

Stage 3: Data Loading (Storage): Delivering processed data to its final destination:

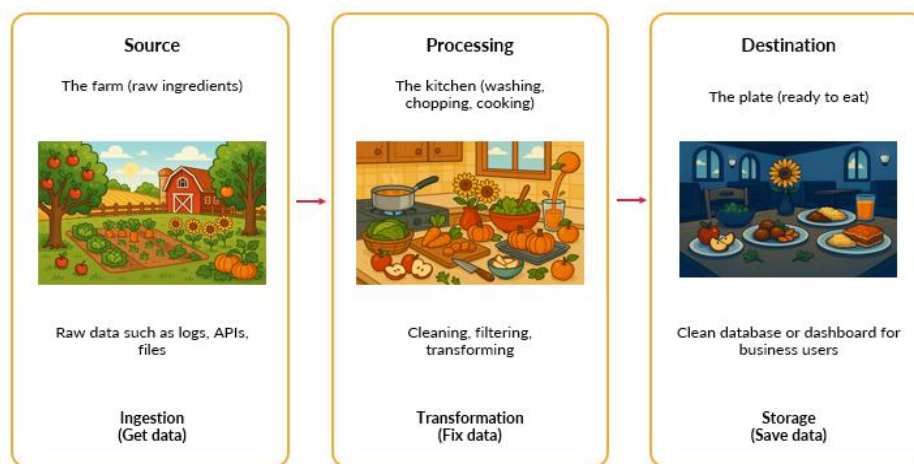
- Data warehouses (Snowflake, Redshift, BigQuery)
- Data lakes (S3, HDFS, Azure Data Lake)
- Operational databases (for application use)
- Business intelligence tools (Tableau, Power BI)
- Machine learning platforms
- Real-time dashboards and reporting systems

Below image illustrates the core ETL (Extract, Transform, Load) pipeline flow, visualizing the standard data pipeline stages.



Data Pipeline Stages Through Analogy

To make these concepts more intuitive, consider this kitchen analogy below:



Just as you wouldn't serve unwashed vegetables directly from the farm, you shouldn't send raw, unprocessed data to business users. The pipeline ensures data is cleaned, prepared, and presentable.

Pipeline Stage	Kitchen Analogy	Technical Example
Source	The farm (raw ingredients)	Raw logs, API responses, CSV files



Ingestion	Shopping and delivery	Extracting data from databases/APIs
Processing	The kitchen (washing, chopping, cooking)	Cleaning, filtering, transforming data
Storage	The plate (ready to eat)	Clean database or dashboard

Need for Data Pipelines

Modern organizations generate and consume data at unprecedented scales. Manual data processing is not only inefficient but impossible at today's data volumes. Data pipelines address critical business and technical needs:

Key Business Drivers

1. Consistency:

Imagine your executive dashboard must show yesterday's sales data every morning at 8:00 AM sharp. With data pipelines:

- Data arrives at exactly 8:00 AM every single day without a human clicking "Run"
- No weekends, no holidays, no sick days—the pipeline doesn't sleep
- Business decisions start on time with reliable, fresh data
- SLAs (Service Level Agreements) are automatically met

Real-world impact: A retail company processing 10 million transactions daily can guarantee that store managers see overnight sales by 7:00 AM, enabling same-day inventory decisions.

2. Scalability: Beyond Human Capacity

Manual data processing hits a hard ceiling:

- A data analyst might process 10,000 rows in Excel



- A data pipeline processes 10 million rows in the same time
- As data volume grows 10x, pipeline cost grows minimally
- No additional headcount needed to handle growth

Example: Netflix processes billions of viewing events daily to power recommendations. No human team could manually analyze this data—automated pipelines are the only solution.

3. Reliability: Self-Healing Infrastructure

Data pipelines are built for failure scenarios:

- If a server crashes at 2:00 AM, the pipeline automatically retries
- Failed tasks alert on-call engineers immediately
- Partial failures don't corrupt downstream data
- Historical runs can be backfilled after system recovery

Scenario: An API rate limit is hit during extraction. Instead of failing completely, the pipeline waits 60 seconds and retries automatically—no human intervention needed.

4. Single Source of Truth

Without pipelines, different teams create their own data extracts, leading to:

- Finance reporting \$1.2M revenue while Marketing reports \$1.3M
- Conflicting metrics in board presentations
- Hours wasted reconciling discrepancies
- Loss of trust in data-driven decisions

With a centralized pipeline:

- Everyone queries the same cleaned, validated dataset
- Business logic is applied consistently (e.g., "What counts as a sale?")
- Data governance and compliance are enforced uniformly



- Audit trails track every transformation

Technical Benefits

- **Version Control:** Pipeline code lives in Git, enabling code reviews and rollbacks
- **Testing:** Unit tests, integration tests, and data quality checks run automatically
- **Monitoring:** Metrics track pipeline health, data quality, and SLA compliance
- **Optimization:** Pipelines can be tuned for performance without rewriting from scratch
- **Collaboration:** Teams work on the same codebase with clear ownership

Types of Data Pipelines

Data pipelines are designed based on latency requirements and data processing patterns. Understanding these types helps you choose the right architecture for your use case.

1. **Batch Pipelines:** Batch pipelines collect, process, and operate on large sets of data as a complete unit, typically on a scheduled basis (hourly, daily, weekly).

Characteristics:

- Process data in large chunks at fixed intervals
- High latency (minutes to hours)
- Optimized for throughput over speed
- Lower infrastructure costs
- Simpler to design and maintain

Common Use Cases:

1. **Nightly ETL Jobs:** Loading yesterday's transactions into a data warehouse at midnight
2. **Historical Reporting:** Generating monthly sales reports with complex aggregations



3. **Data Backups:** Copying production databases to archival storage daily
4. **Financial Reconciliation:** Processing end-of-day banking transactions
5. **Data Science Model Training:** Training ML models on weekly historical data
6. **Payroll Processing:** Computing employee salaries every two weeks

Example Scenario:

A retail company runs a batch pipeline every night at 1:00 AM to:

- Extract sales data from 500+ store databases
- Join with inventory, customer, and promotion data
- Calculate daily KPIs (revenue, units sold, profit margins)
- Load results into a data warehouse for morning reports

When to Use:

- Data doesn't need to be real-time
- Complex transformations requiring full datasets
- Cost optimization is a priority
- Historical analysis and trend reporting

2. Micro-Batch Pipelines: Micro-batch pipelines process data in small batches at frequent intervals (every 1-15 minutes), balancing latency and throughput.

Characteristics:

- Hybrid approach between batch and streaming
- Medium latency (minutes)
- Good balance of complexity and performance
- Easier to implement than true streaming



- Can achieve near-real-time insights

Common Use Cases:

1. **Social Media Analytics:** Processing user engagement metrics every 5 minutes
2. **IoT Sensor Data:** Aggregating device readings in 10-minute windows
3. **Near-Real-Time Dashboards:** Updating executive KPIs every 15 minutes
4. **Log Aggregation:** Collecting and parsing application logs in 5-minute batches
5. **E-commerce Metrics:** Tracking hourly conversion rates and cart abandonment

Example Scenario:

A ride-sharing platform runs micro-batch pipelines every 5 minutes to:

- Aggregate completed rides by city and time window
- Calculate average wait times and driver availability
- Update demand forecasting models
- Alert operations teams to supply-demand imbalances

When to Use:

- Need faster insights than daily batch allows
- True streaming infrastructure too complex/expensive
- Tolerating 5–15-minute delays is acceptable
- Processing logs or time-series data

3. **Streaming pipelines** process data continuously as individual events arrive, with minimal latency (sub-second to seconds).

Characteristics:

- Real-time processing of individual events
- Ultra-low latency (milliseconds to seconds)



- Higher infrastructure complexity and cost
- Requires specialized tools (Kafka, Flink, Storm)
- Always-on, never-ending execution

Common Use Cases:

1. **Fraud Detection:** Flagging suspicious credit card transactions instantly
2. **Stock Trading:** Making automated trades based on real-time market data
3. **Content Recommendations:** Updating Netflix/Spotify suggestions as you browse
4. **Real-Time Personalization:** Adjusting website content based on user clicks
5. **Cybersecurity:** Detecting intrusions and anomalies in network traffic
6. **Dynamic Pricing:** Adjusting Uber surge pricing based on real-time demand
7. **Live Dashboard Updates:** Monitoring system health with second-by-second metrics
8. **Social Media Feeds:** Delivering tweets and posts as they're created

Example Scenario:

A financial institution runs streaming pipelines to:

- Ingest credit card transactions as they occur (100,000/second)
- Apply ML fraud detection models within 100ms
- Block suspicious transactions before authorization
- Alert customers via SMS within seconds of detection

When to Use:

- Sub-second latency is critical
- Processing events individually (not in batches)
- Immediate action required (fraud, trading, alerts)



- Real-time monitoring and anomaly detection

Comparison Summary

Aspect	Batch	Micro-Batch	Streaming
Latency	Hours	Minutes	Milliseconds
Complexity	Low	Medium	High
Cost	Low	Medium	High
Data Volume	Very High	Medium-High	Event-by-Event
Use Case	Reports, analytics	Near-real-time metrics	Fraud, trading, alerts
Tools	Airflow, Luigi	Airflow, Spark	Kafka, Flink, Storm

Introduction to Apache Airflow

Apache Airflow is an open-source platform designed to programmatically author, schedule, and monitor data pipelines. Created by Airbnb in 2014 and donated to the Apache Software Foundation in 2016, Airflow has become the de facto standard for workflow orchestration in data engineering.

Why Airflow is Popular

Industry Adoption:

- Used by Airbnb, Netflix, Spotify, Twitter, Uber, and thousands more
- 90% of organizations use Airflow for ETL/ELT operations
- Active community with 2,000+ contributors
- 400+ provider packages for integrations

Key Advantages:



1. **Flexibility:** Pure Python means unlimited extensibility
2. **Rich Ecosystem:** Pre-built operators for AWS, GCP, Azure, Snowflake, and more
3. **Dynamic DAGs:** Generate pipelines programmatically based on configurations
4. **Observability:** Comprehensive UI with logs, metrics, and alerts
5. **Backfilling:** Easily rerun historical data after fixes
6. **Community Support:** Extensive documentation, tutorials, and forums

Directed Acyclic Graphs (DAGs)

In Airflow, every pipeline is represented as a **Directed Acyclic Graph (DAGs)** in a mathematical structure that defines the relationships between tasks.

Breaking Down the Acronym:

- **Directed:** Tasks have a clear flow with explicit dependencies (Task A → Task B)
- **Acyclic:** No loops or circular dependencies (prevents infinite execution)
- **Graph:** A visual map showing how tasks connect and depend on each other

Why DAGs Matter

Dependency Management:

- If Task B depends on Task A, Airflow ensures:
- Task B only runs after Task A succeeds
- If Task A fails, Task B is skipped (preventing bad data propagation)
- Dependencies can be complex: multiple parents, conditional branches

Example Scenario:

```
extract_customers >> clean_customers >> load_customers
```

```
extract_orders >> clean_orders >> join_with_customers >> load_final_table
```



In this DAG:

- `join_with_customers` waits for both `clean_customers` and `clean_orders`
- If `extract_customers` fails, `load_final_table` never runs
- Parallel execution: `extract_customers` and `extract_orders` run simultaneously

Airflow UI Components

The Airflow web interface consists of several key sections:

DAGs View

Main Dashboard: Think of this as the control panel for all your workflows (DAGs):

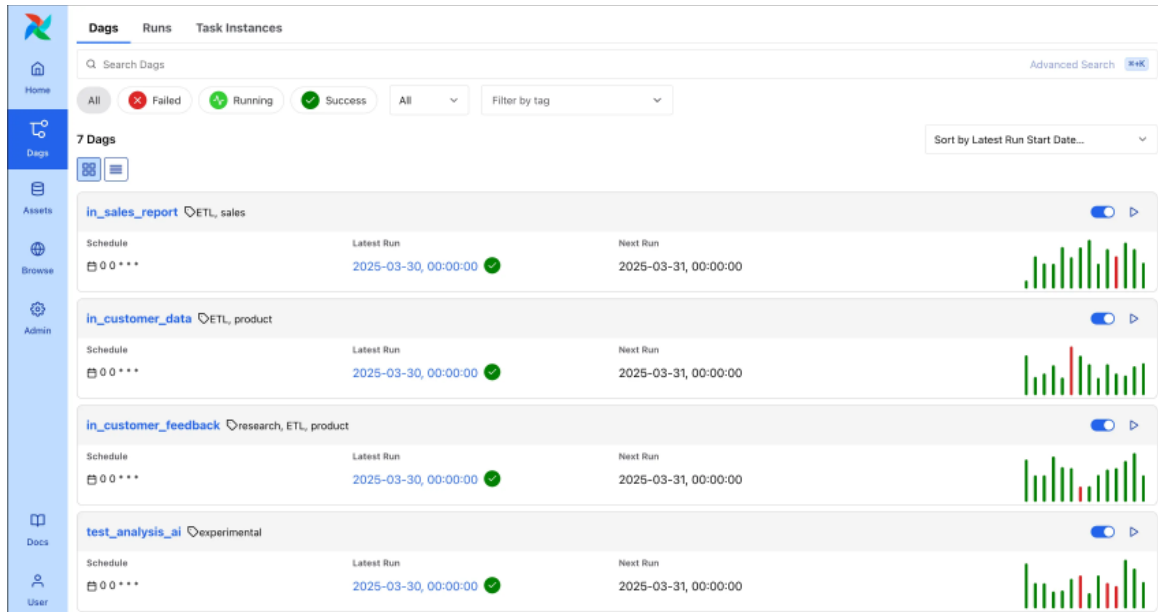
- **List of all DAGs with status indicators** – Shows every DAG you have and whether it's running, paused, or failed.
- **Success/failure statistics** – Quick numbers to see how many runs succeeded or failed.
- **Recent run history** – Shows the last few times each DAG ran and their outcomes.
- **Schedule information** – Tells you when the DAG is set to run next.
- **Quick actions** – Buttons to trigger a DAG immediately, pause it, or delete it.

Filters: Helps you narrow down what you see:

Search by DAG name or tag – Find a specific DAG quickly.

Filter by status – Only show DAGs that are active, paused, or failed.

Following image captures the primary "DAGs" dashboard in the Airflow Web serving as the central command center for all pipeline management.



Datasets View

- Shows **all datasets you've defined**.
- Lets you see **which DAGs depend on which datasets**.
- Helps you **track data lineage**—you know which DAGs produce data and which consume it.

The following screenshot displays the Airflow "Datasets" page from the Web UI, showcasing Airflow's data-aware scheduling feature.

Datasets			
Dataset Events	Dependency Graph	Datasets	
			Sort: Timestamp - Desc
2024-07-23, 11:32:09 EDT	Source: s3://consuming_2_task/dataset_other_unknown.txt	Source:	
2024-07-23, 11:32:03 EDT	Source: s3://unrelated/this-dataset-doesnt-get-triggered	Source: API	Triggered Dag Runs:
▶ { ... } 16 items			Copy
2024-07-22, 21:32:01 EDT	Source: s3://consuming_1_task/dataset_other.txt	Source:	
2024-07-22, 21:32:01 EDT	Source: s3://consuming_2_task/dataset_other_unknown.txt	Source:	
2024-07-22, 21:32:01 EDT	Source: s3://consuming_2_task/dataset_other_unknown.txt	Source:	
2024-07-22, 21:31:55 EDT	Source: s3://dag1/output_1.txt	Source:	Triggered Dag Runs:
2024-07-22, 21:31:50 EDT	Source: s3://output/1.txt	Source:	
▶ { ... } 1 item			Copy
2024-07-22, 21:31:50 EDT	Source: s3://output/1.txt	Source:	
▶ { ... } 1 item			Copy
2024-07-21, 23:29:28 EDT	Source: s3://consuming_2_task/dataset_other_unknown.txt	Source:	
2024-07-21, 23:29:28 EDT	Source: s3://consuming_2_task/dataset_other_unknown.txt	Source:	

DAG Id: dataset_produces_1
Task Id: producing_task_1
Overall Status: success
Started: 2024-07-22, 21:31:49 EDT
Ended: 2024-07-22, 21:31:55 EDT
Duration: 00:00:05

Operators

In Airflow, a **task** is an action you want to do. An **operator** is the **type of task**.

- Built-in operators – Provided by Airflow itself. (Python, Bash, SQL, etc.)
- Provider operators – Provided by external Airflow provider packages to interact with cloud services. (AWS, GCP, Azure, etc.)
- Custom operators – Created by your team or organization to perform tasks specific to your environment or business logic.

Below is an Airflow DAG example demonstrating how to use a PythonOperator to run a Python function as a task.



```
localhost:8080/dags/example_python_operator/code

Airflow DAGs Datasets Security Browse Admin Docs

28 import tempfile
29 import time
30 from pprint import pprint
31
32 import pendulum
33
34 from airflow import DAG
35 from airflow.decorators import task
36
37 log = logging.getLogger(__name__)
38
39 PYTHON = sys.executable
40
41 BASE_DIR = tempfile.gettempdir()
42
43 with DAG(
44     dag_id='example_python_operator',
45     schedule=None,
46     start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
47     catchup=False,
48     tags=['example'],
49 ) as dag:
50
51     # [START howto_operator_python]
52     @task(task_id="print_the_context")
53     def print_context(ds=None, **kwargs):
54         """Print the Airflow context and ds variable from the context."""
55         pprint(kwargs)
56         print(ds)
57         return 'Whatever you return gets printed in the logs'
58
59     run_this = print_context()
60     # [END howto_operator_python]
61
62     # [START howto_operator_python_render_sql]
63     @task(task_id="log_sql_query", templates_dict={"query": "sql/sample.sql"}, templates_exts=[".sql"])
```

Jobs and Logs

1. Jobs: Think of a job as the engine running Airflow. It's the system process that makes tasks happen.

What you can do in the Jobs section:

- **View currently running tasks** – See which tasks are actively running.
- **See queued tasks** – Check which tasks are waiting to run because resources are busy.
- **Monitor resources** – See how many workers or slots are being used.
- **Track scheduler health** – Ensure the scheduler is running smoothly and triggering tasks on time.

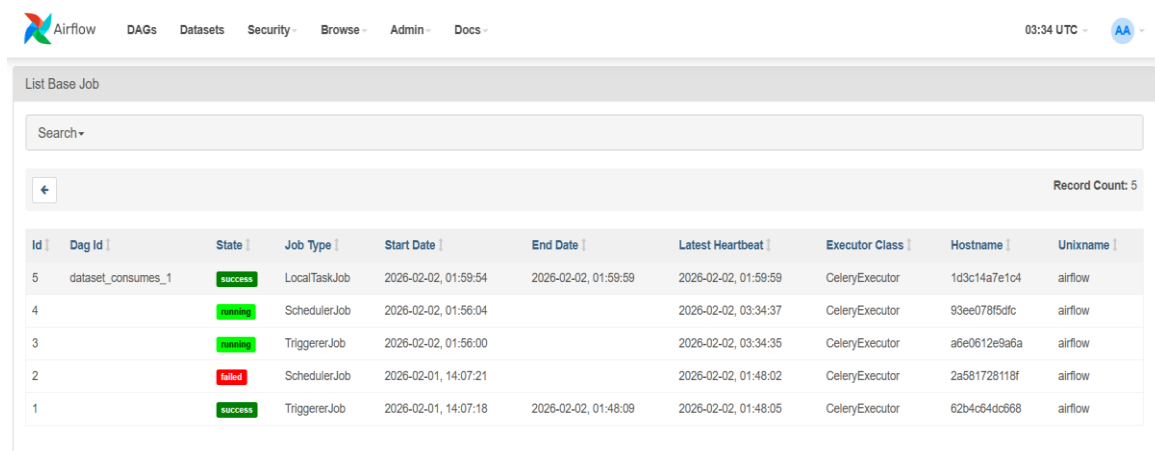
2. Logs : They are like a diary of what happened when a job or task ran.

What you can do in the Logs section:

- **Real-time task logs** – Watch what a task is doing right now.

- **Historical logs** – See what happened in past runs.
- **Scheduler & webserver logs** – Check system-level messages.
- **Download & search logs** – Save logs or search for errors easily.

Below screenshot shows the Airflow Web UI's "Jobs and Logs" page, a critical interface for monitoring pipeline execution and troubleshooting failures



Id	Dag Id	State	Job Type	Start Date	End Date	Latest Heartbeat	Executor Class	Hostname	Unixname
5	dataset_consumes_1	success	LocalTaskJob	2026-02-02, 01:59:54	2026-02-02, 01:59:59	2026-02-02, 01:59:59	CeleryExecutor	1d3c14a7e1c4	airflow
4		running	SchedulerJob	2026-02-02, 01:56:04		2026-02-02, 03:34:37	CeleryExecutor	93ee078f5dfc	airflow
3		running	TriggererJob	2026-02-02, 01:56:00		2026-02-02, 03:34:35	CeleryExecutor	a6e0612e9a6a	airflow
2		failed	SchedulerJob	2026-02-01, 14:07:21		2026-02-02, 01:48:02	CeleryExecutor	2a581728118f	airflow
1		success	TriggererJob	2026-02-01, 14:07:18	2026-02-02, 01:48:09	2026-02-02, 01:48:05	CeleryExecutor	62b4c64dc668	airflow

Conclusion

1. Key Takeaways

Data pipelines form the foundation of modern data infrastructure, enabling organizations to process data at scale with reliability and consistency. Apache Airflow has emerged as the industry-standard orchestration platform, offering:

- **Flexibility:** Python-based "workflows as code" approach
- **Reliability:** Built-in retry mechanisms and error handling
- **Scalability:** From single-machine to distributed Kubernetes deployments
- **Observability:** Rich UI with comprehensive monitoring and logging
- **Extensibility:** 400+ provider packages and custom operator support



2. Choosing the Right Pipeline Type

- **Batch:** For historical reporting, complex transformations, cost optimization
- **Micro-Batch:** For near-real-time dashboards, IoT data, log aggregation
- **Streaming:** For fraud detection, trading, personalization, real-time alerts

3. Airflow's Position in the Ecosystem

While Airflow excels at batch and micro-batch orchestration, it integrates with streaming platforms (Kafka, Flink) to trigger downstream batch processing. Modern data architectures often combine:

- Streaming ingestion (Kafka)
- Batch transformation (Airflow + Spark)
- Data warehousing (Snowflake, BigQuery)
- Visualization (Tableau, Looker)

Additional Resources

Official Documentation:

- Apache Airflow Documentation: <https://airflow.apache.org/docs/>

Community Resources:

- Airflow Slack Community: <https://apache-airflow.slack.com>
- Airflow GitHub Repository: <https://github.com/apache/airflow>

Disclaimer: All content and material on the upGrad website is copyrighted material, belonging to either upGrad or its bonafide contributors, and is purely for the dissemination of education. You are permitted to access, print, and download extracts from this site purely for your own education only and on the following basis:



- *You can download this document from the website for self-use only.*
- *Any copies of this document, in part or full, saved to disc or to any other storage medium, may be used for subsequent, self-viewing purposes or to print an individual extract or copy for noncommercial personal use only.*
- *Any further dissemination, distribution, reproduction, or copying of the content of the document herein, or the uploading thereof on other websites, or use of the content for any other commercial/unauthorized purposes in any way that could infringe the intellectual property rights of upGrad or its contributors, is strictly prohibited.*
- *No graphics, images, or photographs from any accompanying text in this document will be used separately for unauthorized purposes.*
- *No material in this document will be modified, adapted, or altered in any way.*
- *No part of this document or upGrad content may be reproduced or stored on any other website or included in any public or private electronic retrieval system or service without upGrad's prior written permission.*
- *Any rights not expressly granted in these terms are reserved.*